

```

@task
def extract(latitude, longitude, days=60):
    """Call Open-Meteo for the past `days` days of daily weather."""
    params = {
        "latitude": float(latitude),
        "longitude": float(longitude),
        "past_days": days,
        "forecast_days": 0, # only past weather
        "daily": [
            "temperature_2m_max",
            "temperature_2m_min",
            "precipitation_sum",
            "weather_code",
        ],
        "timezone": "America/Los_Angeles",
    }
    response = requests.get(URL, params=params, timeout=30)
    response.raise_for_status()
    return response.json() # a dict, so it can be passed through XCom

```

```

@task
def transform(data):
    """Turn the API response into a list of rows (JSON-serializable for XCom)."""
    lat = data["latitude"] # grid cell the API actually used
    lon = data["longitude"]
    daily = data["daily"]

    records = []
    for i in range(len(daily["time"])):
        record = [
            lat,
            lon,
            daily["time"][i], # 'YYYY-MM-DD'
            daily["temperature_2m_max"][i],
            daily["temperature_2m_min"][i],
            daily["precipitation_sum"][i],
            daily["weather_code"][i],
        ]
        if None in record:
            raise ValueError(f"Missing value on {daily['time'][i]}: {record}")
        record[6] = int(record[6])
        records.append(record)
    return records

```

```
@task
def load(records, target_table):
    cur = return_snowflake_conn()

    # DDL auto-commits in Snowflake, so create the table BEFORE the transaction
    cur.execute(f"""
        CREATE TABLE IF NOT EXISTS {target_table} (
            latitude      NUMBER(9, 6) NOT NULL,
            longitude     NUMBER(9, 6) NOT NULL,
            date          DATE          NOT NULL,
            temp_max      FLOAT         NOT NULL,
            temp_min      FLOAT         NOT NULL,
            precipitation  FLOAT         NOT NULL,
            weather_code  INTEGER       NOT NULL,
            PRIMARY KEY (latitude, longitude, date)
        )
    """)
```

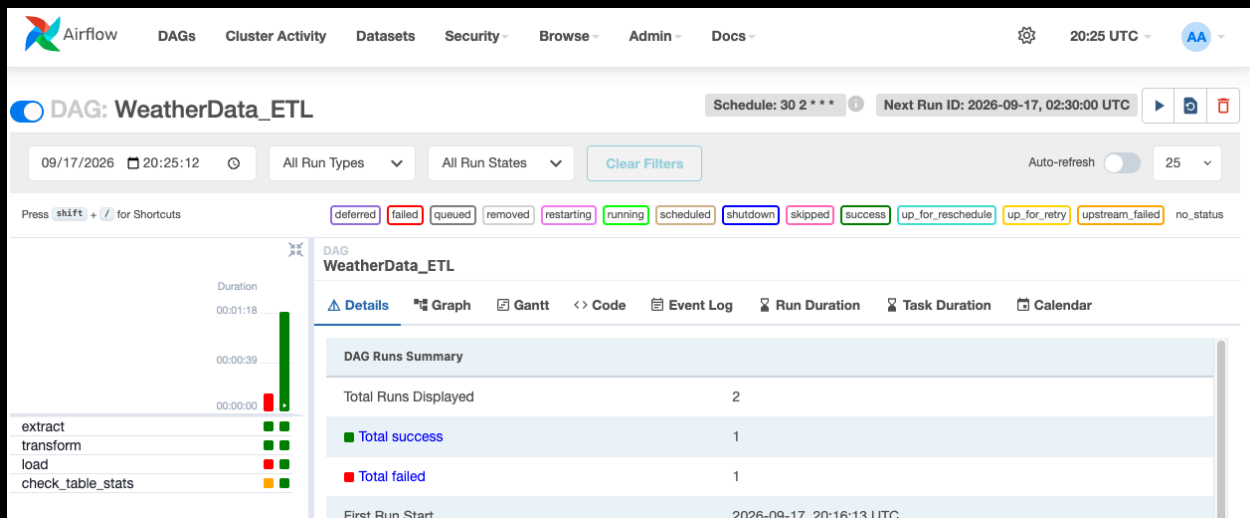
```
# Full refresh: DELETE + INSERT succeed or fail together
try:
    cur.execute("BEGIN;")
    cur.execute(f"DELETE FROM {target_table}")
    sql = f"""INSERT INTO {target_table}
        (latitude, longitude, date, temp_max, temp_min, precipitation, weather_code)
        VALUES (%s, %s, %s, %s, %s, %s, %s)"""
    for r in records:
        print(r)
        cur.execute(sql, r)
    cur.execute("COMMIT;")
except Exception as e:
    cur.execute("ROLLBACK;")
    print(e)
    raise e
finally:
    cur.close()
```

@task

```
▼ def check_table_stats(target_table):
    """Print row count and date range so the log shows the result of the load."""
    cur = return_snowflake_conn()
    try:
        cur.execute(f"SELECT COUNT(*), MIN(date), MAX(date) FROM {target_table}")
        count, min_date, max_date = cur.fetchone()
        print(f"{target_table}: {count} rows, {min_date} ~ {max_date}")
        return count
    finally:
        cur.close()
```

List Variable				
Search▼				
+ Actions▼ ←				Record Count: 2
☐	Key ↕	Val ↕	Description ↕	Is Encrypted ↕
☐	   latitude	37.3382		False
☐	   longitude	-121.8863		False

Connection Id *	snowflake_conn
Connection Type *	Snowflake x Connection Type missing? Make sure you've installed the corresponding Airflow Provider Package.
Description	
Schema	RAW
Login	rahlshah88
Password	snowflake password
Extra	<pre>{ "account": "OVLLRIT-ZG33510", "warehouse": "DEMO_WH", "database": "DEMO_DB", "role": "ACCOUNTADMIN", "private_key_file": "/opt/airflow/keys/rsa_key.p8" }</pre>
Account	OVLLRIT-ZG33510
Warehouse	DEMO_WH
Database	DEMO_DB
Region	snowflake hosted region
Role	ACCOUNTADMIN
Private key (Path)	/opt/airflow/keys/rsa_key.p8



Idempotency

